

ELG/GNG/DTI 5902 Industry Internship Project

Final Report

Student name: David Talson

Graduate Program: Master of Engineering: Electrical and Computer Engineering, Concentration
in Applied Artificial Intelligence

Term: Summer 2023

Project Title: Target Drone Detection with Autonomous Landing

Table of Contents:

1. Introduction.....	7
1.1 Problem Definition.....	7
1.2 Background	7
1.3 Project Context.....	8
2. Design Overview.....	9
2.1 Requirements	10
2.2 Detailed Design.....	10
2.2.1 System Tools.....	10
2.2.2 System Setup.....	14
2.2.3 Drone Control and Stabilization	15
2.2.3.1 Mission Server Model.....	15
2.2.3.1.1 Mission Server Position Control.....	15
2.2.3.1.2 Communication	16
2.2.3 Drone Search.....	18
2.2.4 Object Detection	19
2.2.4 Position Estimation	21
2.2.5 Drone Landing	22
2.3 Implementation.....	22
2.4 Testing.....	24
2.4.1 Data Plan.....	24
2.4.2 Validation and Verification.....	24
3. Overall Results and Analysis	27
3.1 Overall Results	27
3.2 Analysis.....	28
3.3 Object Detection	29
3.4 Improvement Opportunities	29
3.5 Learning Outcomes and Career Objectives	29
4. Deployment Plan	30
4.1 Requirements	30
4.2 QDrone Power Up and Connectivity	30
4.3 Mission_Server_DT Simulink Model.....	30
4.4 Command Stabilizer Simulink Model	32
4.5 Object_Position_Estimation Script	33
5. Conclusion and Future Work	34
6. References.....	35

Table of Figures:

Figure 1: Conceptual Design of System	8
Figure 2: Quanser QDrone	9
Figure 3: FrSky Flight Controller	10
Figure 4: Indoor Area with OptiTrack Cameras Highlighted in Red Boxes	10
Figure 5: Camera Calibration with CW-500 Wand	11
Figure 6: CS-200	11
Figure 7: QDrone Localization Markers	11
Figure 8: Object Tracking in Motive Software	12
Figure 9: Netgear Nighthawk R7000	12
Figure 10: System Setup	13
Figure 11: Mission Server Simulink Model.....	13
Figure 12: Mission Server Position Control	14
Figure 13: Mission Server Communication System	15
Figure 14: Command Stabilizer Simulink Model	16
Figure 15: Communication System	16
Figure 16: Grid Search Simulink Block.....	17
Figure 17: Grid Search Algorithm Flowchart	18
Figure 18: Object of Interest	18
Figure 19: Annotated Image	18
Figure 20: Faster RCNN Detector	19
Figure 21: Training Result of Faster RCNN Model	19
Figure 22: Detected Object in Test Image	19
Figure 23: Position Estimation of Detected Object	20
Figure 24: QDrone Landing System	20
Figure 25: System Implementation	21
Figure 26: System Implementation Architecture	21
Figure 27: Simulation Time vs QDrone Height.....	22
Figure 28: Drone Grid Search.....	23
Figure 29: Object Detector Results	23
Figure 30: Mean Average Precision Plot	24
Figure 31: Actual vs Predicted Object Positions	24
Figure 32: Model Coefficients and Performance (X - Left, Y-Right).....	24
Figure 33: Autonomous Target Detection and Landing	25
Figure 34: Video Transmission System	27
Figure 35: Netgear Router Connected Devices.....	28
Figure 36: System Target File Description	28
Figure 37: Mex File Arguments.....	29
Figure 38: FrSky Controller Switch Positions and USB dongle.....	29
Figure 39: Calibration File Upload	29
Figure 40: Monitor and Tune	30
Figure 41: Grid Search Enable.....	30
Figure 42: System Target File Description	30
Figure 43: Command_Stabilizer Mex-File Arguments	31
Figure 44: QDrone Signals Monitor	31
Figure 45: QDrone Arm and Takeoff Signals.....	32

Table of Tables:

Table 1 : Drone System Integration Key Points 9

Table 2: QDrone Technical Specifications 10

Acronyms

AI: Artificial Intelligence
API: Application Programming Interface
BF: Body Frame
CNN: Convolutional Neural Network
CS-200: Calibration Square 200
CW-500: Calibration Wand 500
DoF: Degrees of Freedom
Faster RCNN: Faster Region-based Convolutional Neural Network
FIFO: First In First Out
HF: Horizon Frame
IF: Inertial Frame
LQR: Linear Quadratic Regulator
mAP: Mean Average Precision
PID: Proportional Integral Derivative
RGB: Red, Green, Blue
ROS: Robot Operating System
SAR: Search and Rescue Operations
SLAM: Simultaneous Localization and Mapping
SVM: Support Vector Machine
UAV: Unmanned Aerial Vehicle
URI: Uniform Resource Identifier
USB: Universal Serial Bus
QDrone: Quanser QDrone
WiFi: Wireless Fidelity
QUARC: QUARC Real-Time Control Software

1. Introduction

In this section, the motivation for this project, its applications, relevant work done in the past and benefits are presented.

1.1 Problem Definition

Canada is the second largest country in the world, spanning over 18 million square kilometers of land and water. According to Public Safety Canada, this presents its unique challenges in Search And Rescue (SAR) operations. Canada's SAR teams respond to over 15,000 calls each year and give assistance to over 25,000 people [1]. Various technologies including drones are being used for this operation. However, during SAR operations, drones used are still manually controlled with operators having to analyse the video feed to make crucial decisions. Identifying the need for improvement birthed the idea of this project. The goal of this project is to design a prototype version of a drone system capable of scanning an area, detecting an object, estimating its position and landing on it. This will greatly advance the effectiveness and efficiency of SAR operations. Also, multiple drones can be deployed simultaneously which means vast areas can be searched efficiently to locate individuals in distress and possibly bring survival kits before a rescue team arrives.

Through this project, knowledge will be gained on drone control algorithms, grid search algorithms, object detection design, and position estimation algorithms. This will also lead to contributions in the development of applications of drone systems.

1.2 Background

There has been some very good work with regards to drone detection, tracking, and landing. A comprehensive survey was done on previous work to gain insights on the latest advancements in this area. The papers reviewed were both academic and industry papers in nature. The following summarizes key findings from the papers surveyed:

Academic

Pose Estimation Method for Autonomous Landing of Quadrotor Unmanned Aerial Vehicle: This paper explores the design of a landing sign which can be detected by a drone at different heights. The sign was designed to provide the drone with information which will enable the drone to estimate the position of the sign [2]. The landing sign comprised of a circle and three squares. At higher altitudes, the drone detects the circle and at lower altitudes, the drone detects the corners of the square. The detection mechanism used was derived using image processing techniques. After the landing sign is detected, the real-world coordinates are calculated using Perspective-Three-Point (P3P) algorithm [3].

Object Pose Estimation for UAV Navigation Using an End-to-end Lightweight CNN: This paper explores the development of a lightweight convolutional neural network (CNN) to estimate the position of an object detected from an unmanned aerial vehicle (UAV) [4]. The CNN comprised of a 5x5 convolutional layer with max pooling of size 3x3, 3x3 kernel 32-, 64-, and 64-dimension residual blocks, a dropout layer and a flatten layer [5]. The object is detected and pose estimated by the CNN model. This was achieved by training the CNN model with Red, Green, Blue (RGB) images with the object captured from the camera of the UAV, the spherical coordinates of the camera in the frame, and yaw angle of the gate. This enables the CNN model to learn a system to estimate the pose of a detected object.

Industry

Quadrotor UAV Flight Control with Integrated Mapping and Path Planning Capabilities: This is a thesis where a robot operating system (ROS) in an isolated environment for path planning of a Quadrotor was developed. Multiple quadrotor laws were implemented using pre-defined specifications [6]. A Proportional Integral Derivative (PID), Linear Quadratic Regulator (LQR), Backstepping and Adaptive Control was implemented to tackle motor actuation loss. Also, a laser-based Simultaneous Localization and Mapping (SLAM) was mapping, localization, path planning, and obstacle avoidance.

Integrated Path Planning and Trajectory Tracking Control for Quadrotor UAVs with Obstacle Avoidance in the Presence of Environmental and Systematic Uncertainties: Theory and Experiment: This paper develops a control framework for path planning and trajectory tracking of a quadrotor. A search algorithm which is used to plan an optimal path while avoiding static and dynamic obstacles was developed [7]. Also, and adaptive sliding trajectory controller was developed to account for uncertainties and improve tracking performance.

1.3 Project Context

The external systems which will be interacted with are listed below:

MATLAB – Simulink: The drone control models will be designed, built, and deployed using Simulink. Simulink is a Matlab tool within Matlab used for advanced model design and simulation.

QUARC Real-Time Control Software: QUARC Real-Time Control Software (QUARC) will be used for the live deployment of the drone. QUARC is part of the Quanser Research studio which offers the test setup and environment where the system will be run.

Quanser QDrone The Quanser QDrone is a carbon fiber quadrotor drone which will be used for this project, it consists of three cameras, an accelerometer, a gyroscope and has Wireless-Fidelity (WiFi) capabilities. Also, it has a high-powered processor optimized for real time applications.

OptiTrack: OptiTrack which is a motion capture system created by NaturalPoint. This is used to provided real world positional information of objects.

External Contacts: Quanser Support Team: Support will be provided by the Quanser customer support team. Quanser has provided a lot of demo videos and guides for various experiments and applications.

2. Design Overview

The conceptual design of this project is to design a drone system where the drone will autonomously search an area using a grid search pattern, identify an object using an object detection model, estimate the real-world position of the detected object and land on the detected object using a landing algorithm. The figure below shows an architectural view of the system:

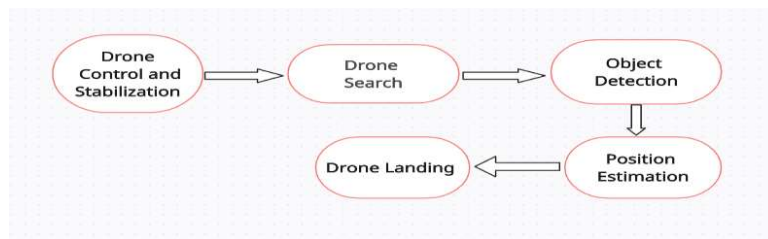


Figure 1: Conceptual Design of System

As can be seen from the figure above the system consists of 5 main blocks which are explained below:

Drone Control and Stabilization: The goal of this block is to design a drone flight controller that provides stable flight at a constant altitude while moving in various directions. This means the drone flight controller and flight commands must be programmed to do this. The drone should have positional awareness also for accurate navigation.

Drone Search: The drone will scan a specified area iteratively until an object is detected. This will ensure the area is scanned accurately and comprehensively.

Object Detection: The drone system will detect objects from the video feed of the drone using object detection Artificial Intelligence (AI) models. Detected objects will be visually highlighted on the output video using a bounding box.

Position Estimation: The real-world coordinates of a detected object will be estimated from the image of the detected object. This will be the landing coordinates of the drone.

Drone Landing: The drone will be programmed to autonomously navigate to the landing coordinates and land safely.

As can be seen from the blocks above, integration is an important part of this system as data will be shared in real time to make decisions. The table below shows key integration points of the system:

Table 1: Drone System Integration Key Points

Integration Point	Data Shared
Drone Control and Stabilization <-> Drone Search	Drone Current Position, Drone Desired Position
Object Detection <-> Position Estimation	Detected Object Bounding Box
Position Estimation <-> Drone Search	Drone Current Position, Drone Landing Position
Drone Landing <-> Drone Control and Stabilization	Drone Landing

2.1 Requirements

The requirements of the system are listed below:

1. Implement drone autonomous control and stabilization at a specified altitude
2. Implement drone comprehensive area search at constant altitude and speed
3. Perform object detection on video feed from drone in real time
4. Detect objects in video and draw bounding boxes on detected objects
5. Estimate landing position of drone when object is detected
6. Land safely on object
7. System should operate in real-time

2.2 Detailed Design

In this section, a detailed design of the system is provided.

2.2.1 System Tools

Quanser QDrone: The QDrone (Qdrone) autonomous air vehicle is a mid-sized quadrotor equipped with high-resolution cameras and built-in WiFi capabilities [8]. This device is suitable for research applications as it is made of carbon-fiber frame which is highly manoeuvrable and can withstand high impact [9]. The powerful on-board processors and cameras make it suitable for streaming and real-time monitoring. The QDrone uses an open software architecture which allows for the design and deployment of algorithms through QUARC. The figure below shows an image of the QDrone.



Figure 2: Quanser QDrone

As can be seen from figure 2 above the QDrone is ideal for a wide range of applications. The table below shows some technical specifications of the QDrone:

Table 2: QDrone Technical Specifications

Dimensions	40 x 40 x 15 cm
Weight (with battery)	~850 g
Max Payload	~300 g
Onboard Computer	Intel® Aero Compute Board Quad-core 64-bit 2.56 GHz processor 4 GB LPDDR3-1600 RAM
Cameras Intel® RealSense™ (R200) Omnivision OV7251	Intel® Aero Vision Accessory Kit Depth sensing (3-4 metre range) Vision (640x480 @ 60 FPS or 1080p @ 30FPS) VGA (640x480 @ 120 FPS)

As can be seen from the technical specifications of the QDrone in the table above, it is equipped to execute complex algorithms and processing.

FrSky Taranis X9 Lite S: This flight controller joystick is used to provide the QDrone with an emergency stop and takeoff/land signals. This emergency stop switch stops the QDrone immediately and the takeoff/land switch can be used to take off or land the drone safely. The figure below shows the flight controller:



Figure 3: FrSky Flight Controller

The main purpose of the flight controller will be to land the QDrone in cases of emergency.

OptiTrack Motion System: OptiTrack is a technology used for motion capture in various applications ranging from robotics, animation, virtual reality, and production. OptiTrack provides 6 Degrees of Freedom (DoF) tracking for ground and aerial objects using high-speed cameras and reflective localization markers. Also, OptiTrack provides support for both indoor and outdoor scenarios. For an indoor scenario which is applicable to this project, a comprehensive camera coverage and calibration. The image below shows the area with camera coverage.



Figure 4: Indoor Area with OptiTrack Cameras Highlighted in Red Boxes

As can be seen from figure 4 above, the cameras are used to track objects in the indoor area. The camera calculates the real time position of objects by measuring the reflected light from the localization marker. In order to define an object to be tracked, the object must have localization markers and be defined as a rigid body. OptiTrack uses a software called Motive which is used to calibrate the cameras and define rigid bodies. In order to setup an object for real-time tracking, the following have to be performed:

Camera Calibration: The cameras are calibrated using a Calibration Wand 500 (CW-500). CW-500 is a calibration wand of 500mm. This wand has 3 localization markers which are continually waved in front of the cameras slowly and in a spiral motion. This enables the cameras

to capture data points and calculate their respective positions to each other. The image below shows the CW-500 wand:



Figure 5: Camera Calibration with CW-500 Wand

As can be seen from figure 5 above, the cameras are being calibrated. A minimum of 20,000 samples for each camera was collected. Then a calibration file was generated and exported.

Ground Plane and Origin Definition: After camera calibration, the ground plane is defined using the Calibration Square 200 (CS-200). The CS 200 is placed in the indoor area and the markers highlighted. Setting the ground plane informs the software of the coordinate system to use. Various placements of the square would give different x,y, and z axis. The figure below shows the calibration square in the indoor area:

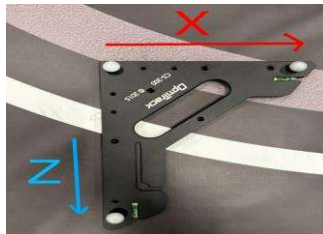


Figure 6: CS-200

As can be seen from figure 6 above, the CS-200 defines the origin of the indoor area and x,y,z axis in the software.

Rigid Body Definition: The rigid body to be tracked must be placed in the indoor area and defined as rigid bodies for tracking. The rigid body should have at least 3 localization markers attached securely to the object [10]. This enables the camera to give the 6 DoF of the object. The image below shows the QDrone with the localization markers highlighted in red boxes:

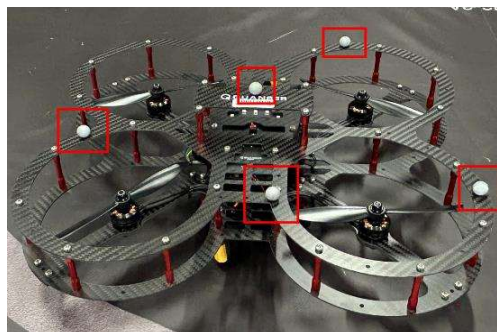


Figure 7: QDrone Localization Markers

As can be seen from figure 7 above, the QDrone is trackable by the motive software.

After calibration and defining the QDrone as a rigid body, when the QDrone is placed in the indoor area, the motive software tracks the object as shown in the in image below:

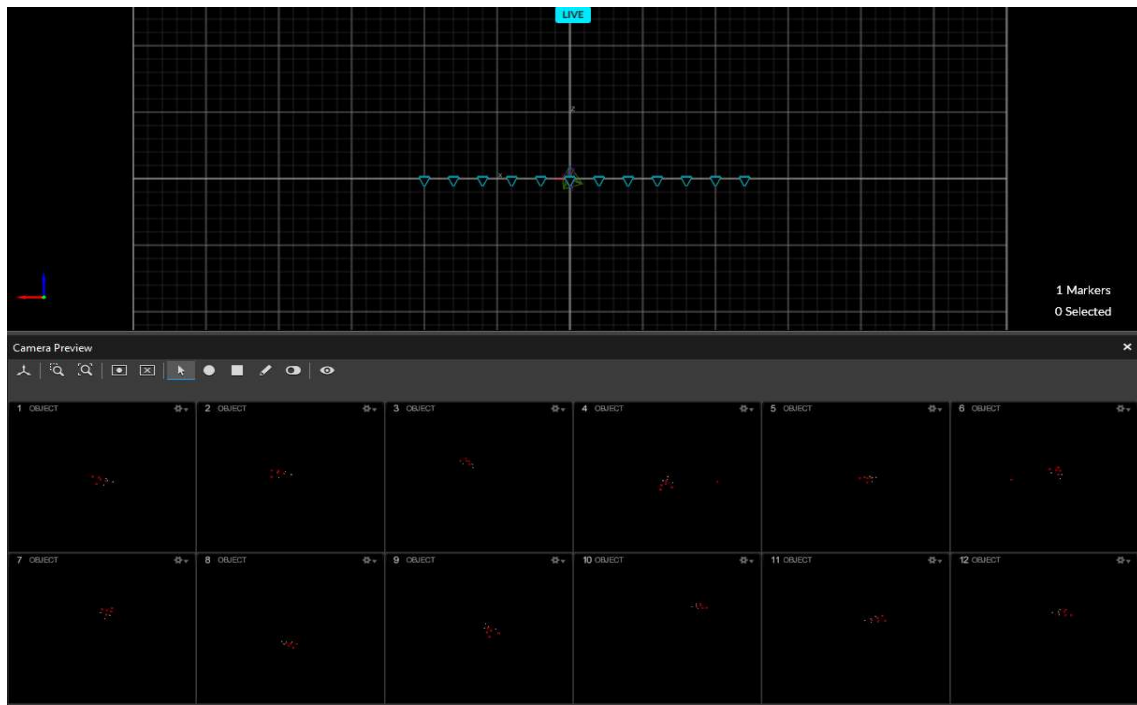


Figure 8: Object Tracking in Motive Software

As can be seen from figure 8 above, the object is being tracked. OptiTrack Motive also has an Application Programming Interface (API) which enables other software to read positional data of objects in the indoor space.

QUARC Real-Time Control Software: QUARC is a real time control software designed by Quanser which allows for the design, development, and deployment of real-time application on various hardware through Simulink [11]. QUARC consists of libraries for communications, multi-threaded execution, image and video processing [12]. This allows for seamless design, control, and deployment of control algorithms to external hardware.

Netgear Nighthawk R7000: The Netgear Nighthawk R7000 is a router used as a means of communication between the QDrone and the host machine. The QDrone and host machine are connected to the router. After a model is designed in simulink, the model is deployed to the QDrone through the router. Also, data streaming and communication is done through the router. The figure below shows the router:



Figure 9: Netgear Nighthawk R7000

2.2.2 System Setup

A block diagram of the system setup is shown below:

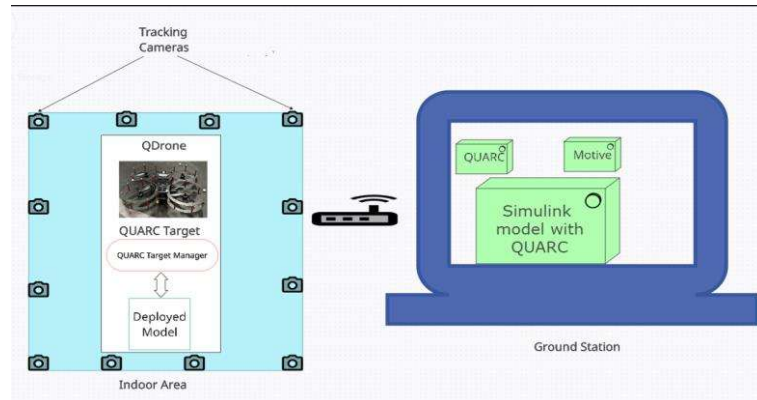


Figure 10: System Setup

As can be seen from figure 10 above the system setup to develop this project is shown. The ground station is a computer equipped with QUARC, MATLAB, and Simulink installed. Also, the motive software is installed. The motive software reads the object positions from the cameras via Universal Serial Bus (USB) connections. The QDrone also has a QUARC manager for running the deployed models. After a model is built, QUARC deploys it to the target through the router for execution.

2.2.3 Drone Control and Stabilization

Quanser's support team provided two Simulink models for the QDrone stabilization at a height of 1m. These models are the Mission Server and Command Stabilizer. Some parts of the Simulink models are out of the scope of this project and only the parts used for this project are discussed.

2.2.3.1 Mission Server Model

The mission server model is deployed on the host machine as its target. When this model is built and run, it tracks the current and desired position of the QDrone, the emergency stop and takeoff/land signals from the Fr-Sky controller, and it also sends a timestamp which is used to calculate the loopback time of the system. All these signals are sent to a stream server to be received by the QDrone. The image below shows the Mission Server block diagram:

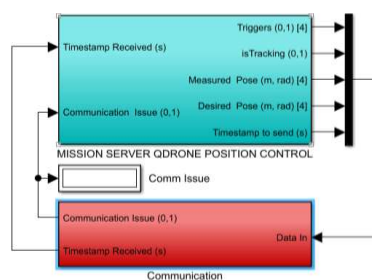


Figure 11: Mission Server Simulink Model

As can be seen from figure 11 above, the subsystems are mission server QDrone position control and communication which are further explained below:

2.2.3.1.1 Mission Server Position Control

The mission server position control subsystem is the control station of the QDrone. The commands for the QDrone are dispatched from here. Also, the takeoff/land commands are

outputted from here. The figure below shows a block diagram of the mission server position control subsystem:

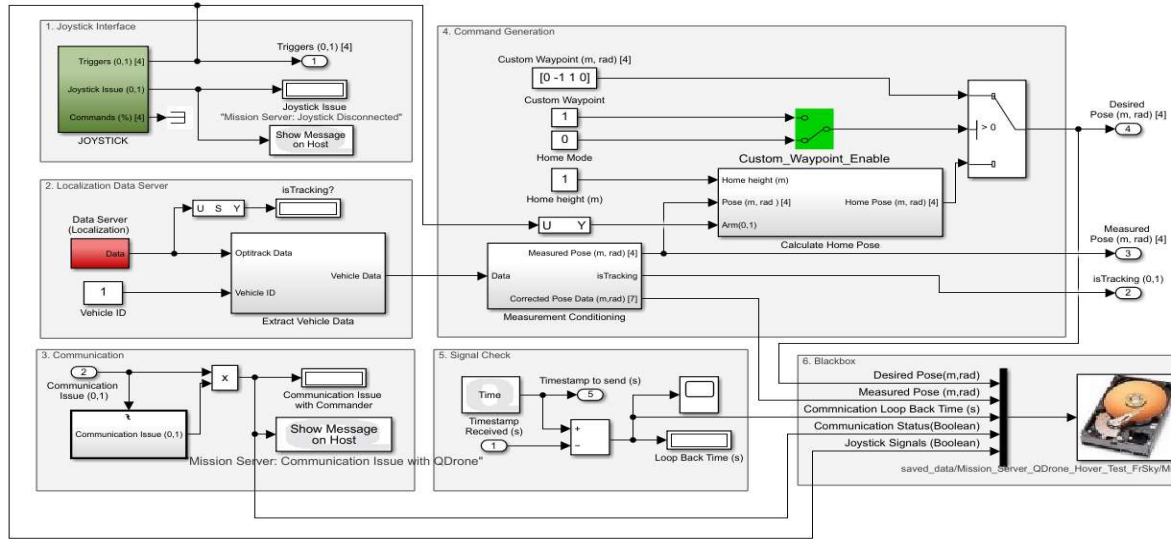


Figure 12: Mission Server Position Control

As can be seen from figure 12 above, there are 5 main sections of the system. Each section serves a different purpose as explained below:

Joystick Interface: This section monitors the input signal from the Fr-Sky controller. The controller is used to send commands to the QDrone to take/land or perform an emergency stop. The commands from the controller are read using a Host Game Controller block from the QUARC which reads the signal from the controller from the host and not the target.

Localization Data Server: This section reads the positional information of the QDrone from the OptiTrack Localization System. This is done using the OptiTrack Trackables block which is a QUARC third-party library that connects to the Motive software API. The position data received from the block is converted from the motive frame to the research studio frame. This is necessary to ensure the pose of the QDrone is calculated accurately. The matrix used for this conversion is shown below:

$$\begin{pmatrix} x_{Research_Studio} \\ y_{Research_Studio} \\ z_{Research_Studio} \end{pmatrix} = \begin{pmatrix} 0 & 0 & -1 \\ -1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x_{Motive_Frame} \\ y_{Motive_Frame} \\ z_{Motive_Frame} \end{pmatrix}$$

As can be seen from the equation above the x and y axis are flipped in the opposite directions. The need for this transformation is to ensure the axes of the Motive Frame is aligned with the Research Studio Frame.

Communication: The communication section monitors the integrity of the data transfer between the QDrone and the Mission Server model. The triggered subsystem is triggered on the falling edge of the communication issue signal. The falling edge was selected because the Mission Server is run before the Command Stabilizer model. This means the communication signal is 1 until the Command Stabilizer model is run which causes the communication signal to go to 0 thereby triggering the subsystem.

Command Generation: As the name implies, the command generation section calculates the measured pose of the QDrone then calculates a desired pose. The positional coordinates of the QDrone received is corrected by adding the positional offset of the QDrone from the ground to derive its actual height from the ground. The ‘Calculate Home Pose’ subsystem receives the home pose from the ‘Measurement Conditioning’ subsystem and arm signal from the Joystick Interface. The ‘Calculate Home Pose’ subsystem only outputs the home when the arm signal is 1. Also, the home height is set to 1m this means the QDrone is set to fly at 1m above the ground at the current location of the QDrone position. The manual switch block is used to switch to a custom defined desired pose for the QDrone. When the switch is set to 1, the ‘Custom Waypoint’ coordinate is outputted as the desired position for the QDrone as it triggers the switch block.

Signal Check: The ‘Signal Check’ section calculates the loopback time between when data is sent to the Commander Stabilizer model and when it is received. The time received from the import is subtracted from the time block which outputs the time since simulation started to calculate the loopback time.

2.2.3.1.2 Communication

The communication system sends data to the Commander Stabilizer model running on the QDrone. The communication system is shown in the figure below:

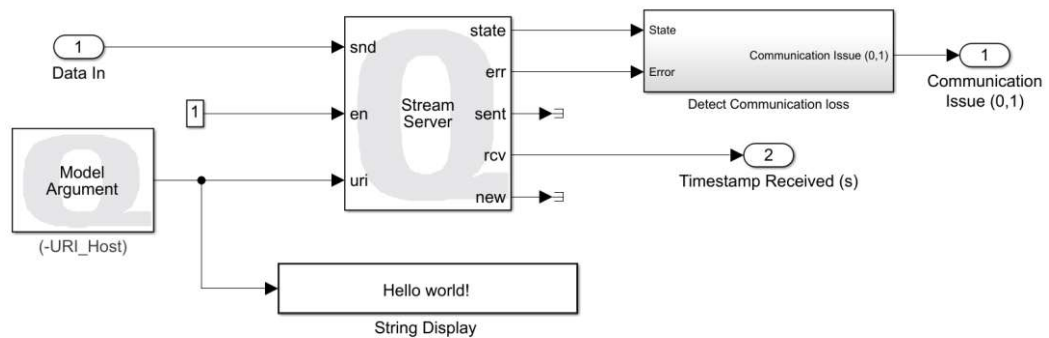


Figure 13: Mission Server Communication System

From figure 13 above, the Stream Server block listens for connections on the Uniform Resource Identifier (URI) of the host machine. When a connection is initialized, data is streamed to the client. The ‘Detect Communication Loss’ subsystem analyses the state and error signals from the stream server. If the signals are above the specified threshold, the communication signal value is set to ‘1’.

2.2.3.2 Commander Stabilizer Model

The Commander Stabilizer model is deployed on the QDrone as its target by QUARC. This model is responsible for flying the QDrone to the desired pose at a constant speed and stabilizes the QDrone at the desired position. The figure below shows the Commander Stabilizer Simulink model:

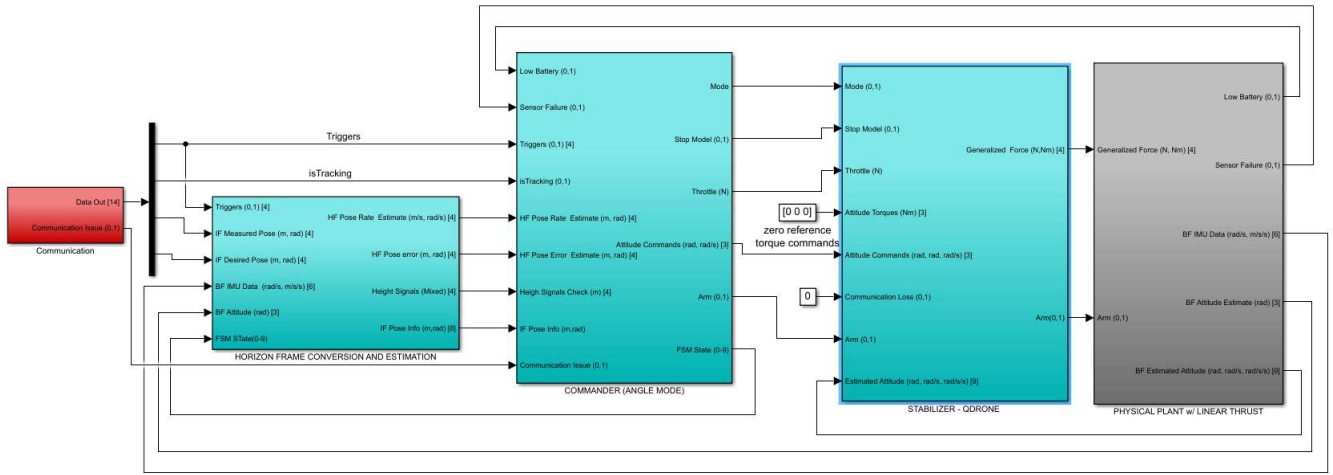


Figure 14: Command Stabilizer Simulink Model

As can be seen from figure 14 above, the Command Stabilizer consists of 5 main subsystems. For the scope of this project only the Communication subsystem was modified, the remaining subsystems was used as provided by the Quanser team.

2.2.3.2.1 Communication: The Communication subsystem connects to the stream server of the Mission Server model and receives data. The Communication subsystem send the loopback time back to the stream server. The figure below shows the communication system:

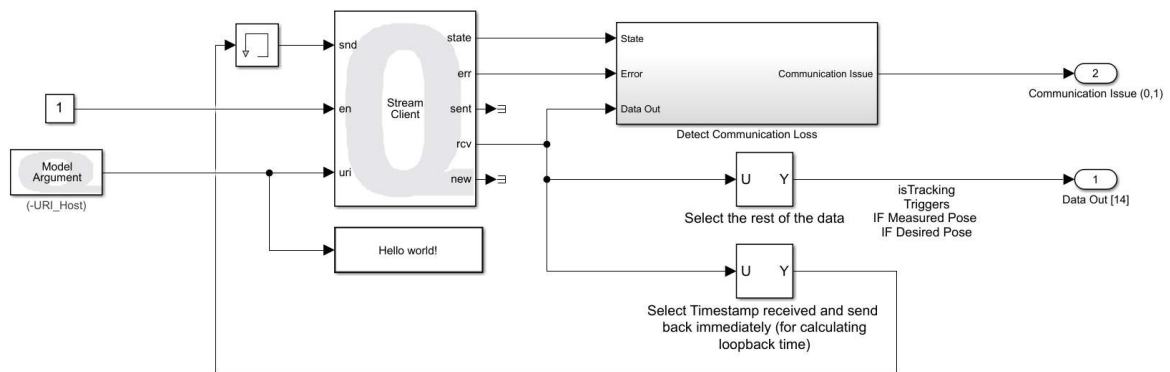


Figure 15: Communication System

From figure 15 above, the Stream Client connects to the URI of the host machine where the stream Server of the Mission Server model would be listening for connections. The 'Detect Communication Loss' subsystem takes the state, error, and received data from the stream client to analyse for communication loss. Communication loss is set to '1' when any of the signals is above the set threshold.

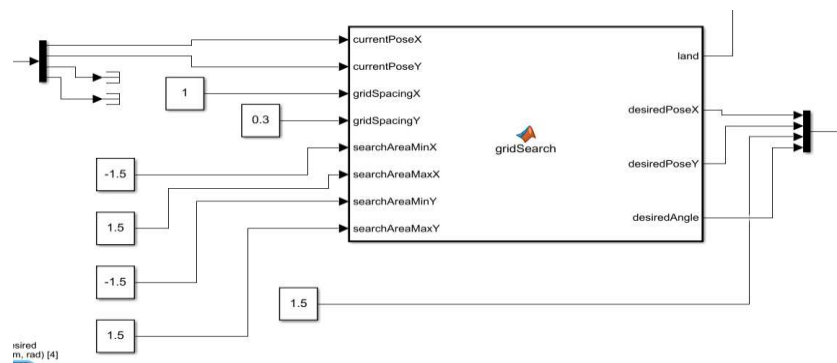
2.2.3 Drone Search

There are various space search algorithms which were explored; however, after comparison, the Grid Search algorithm was selected because it provided the least intensive and comprehensive way to search an indoor area. The table below shows various search algorithms and its advantages and disadvantages:

Table 2: Search Algorithm Comparison

Search Algorithm	Advantages	Disadvantages
Grid Search	- Minimal complexity - Comprehensive area coverage - Relatively easy to implement	- May consume time to search large areas - May miss some areas if grid size is too large
Dijkstra's Algorithm	- Computes shortest path to search area - Suitable for large areas	- Memory intensive - Increased Time complexity
Random Search	- Easy to implement - Minimal complexity	- Some areas in the search area might never be searched - Might go out of desired search area
Spiral Search	- Easy to implement - Minimal complexity	- Corners of search area might be missed

As can be seen from table 2 above, the grid search area is the most optimal for the application of this project as it will provide comprehensive coverage at minimal complexity and computation. However, for the grid search algorithm, a grid size which allows for the comprehensive coverage must be selected. The figure below show the Grid Search block designed for the QDrone:

**Figure 16: Grid Search Simulink Block**

As can be seen from figure 16 above, the grid search block takes the current x and y positions of the drone, the desired grid x and y grid spacing, minimum and maximum search areas in both the x and y directions. These inputs are used to create a grid and the output is the desired x and y position for the QDrone. The flowchart below shows a diagram of the code:

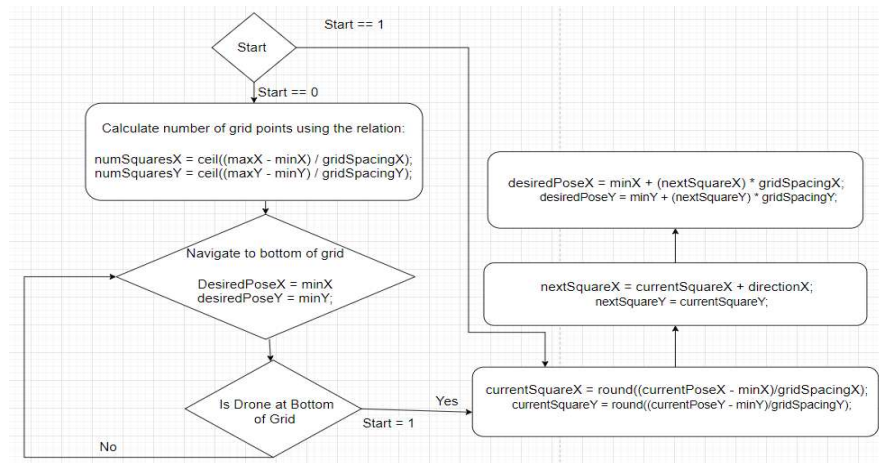


Figure 17: Grid Search Algorithm Flowchart

Figure 17 above shows a simplified flowchart of the grid search algorithm. When the code starts, the number of grid points is calculated and the desired position is set to the search area minimum 'x' and 'y' values respectively where the drone will start the grid search. When the drone is at the desired position the grid search is started where the drone calculates desired position based on the current position and next square. Lastly, although not shown in the flowchart, when the drone gets to the edge it goes to the next 'y' square and moves in the negative 'x' direction. The grid search continues to search the area until it detects an object or it is stopped.

2.2.4 Object Detection

In order to build an object detection model, images of the object of interest must be captured and annotated. These annotated images are then used to train an object detection model. The object of interest for this project is the mouse pad shown below:



Figure 18: Object of Interest

Multiple images which include test and train images of the object were taken in the workspace and annotated using the Image Laber application in MATLAB. The screenshot below shows the image annotation:

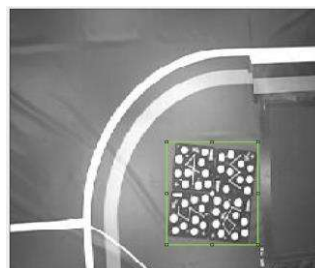


Figure 19: Annotated Image

From 19 above, it shows how captured images from the QDrone were annotated. Around 150 images were captured and annotated. These images were then used to train an object detection model. Different object detection models were evaluated but the Faster Region-based Convolutional Neural Network

(Faster RCNN) was selected as it had the best performance to execution time.

Region-based Convolutional Neural Network (RCNN) is an object detection process where regions which might contain the object are identified then each region is classified using its extracted CNN features. The regions which are identified called region proposals are identified using an algorithm such as Edge Boxes [13]. Each region is treated as an image and classified using the CNN [14]. Lastly, the region proposal boxes are tuned using a Support Vector Machine (SVM). Although RCNN can detect objects, it takes long to execute. This led to the development of Fast RCNN and Faster RCNN detectors. The Faster RCNN detector includes a region proposal network layers to create region proposals instead of using algorithms used in the RCNN detector. This allows for faster execution. The figure below shows the Faster RCNN architecture:

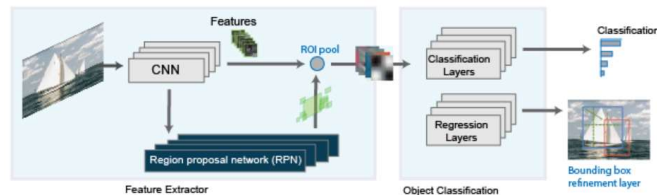


Figure 20: Faster RCNN Detector

As can be seen from figure 20 above, the region proposal network is key. The region proposal network uses anchor boxes of different sizes to estimate possible object locations. The table below shows the performance metrics of the Faster RCNN model after training on captured object images from the QDrone:

Training on single GPU.

Initializing input data normalization.

Epoch	Iteration	Time Elapsed (hh:mm:ss)	Mini-batch Loss	Mini-batch Accuracy	Mini-batch RMSE	RPN Mini-batch Accuracy	RPN Mini-batch RMSE	Base Learning Rate
1	1	00:00:13	1.6105	93.65%	0.18	37.11%	1.02	1.0000e-04
7	50	00:02:14	0.8165	91.67%	0.09	96.06%	0.94	1.0000e-04
10	80	00:03:15	0.5791	94.55%	0.06	96.75%	0.63	1.0000e-04

Figure 21: Training Result of Faster RCNN Model

As can be seen from figure 21 above the model has a high accuracy and when tested on an image it accurately draws the bounding box as shown below:

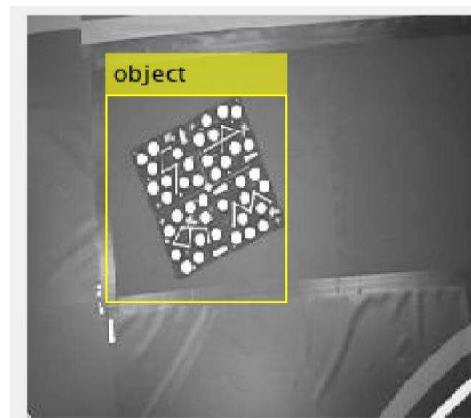


Figure 22: Detected Object in Test Image

As can be seen from figure 22 above, the object was properly detected by the object detection model.

2.2.4 Position Estimation

After the object is detected by the object detection model and the bounding boxes are drawn, the bounding box of the detected object has to be mapped to real world coordinates and the real-world position of the QDrone should be estimated. In order to learn a mapping from the camera images to real world coordinates, a model has to be trained with images from the QDrone where the object's and QDrone's real world 'x' and 'y' coordinates are known. Also, the image is then annotated and the bounding box dimension recorded. This data is then used as a set of features to train a regression model which can estimate the real-world coordinates of a detected object. In order to capture the real-world position of the object some OptiTrack localization markers were placed on the object and images were captured. The figure below shows the estimated position by the regression model after an object was detected:

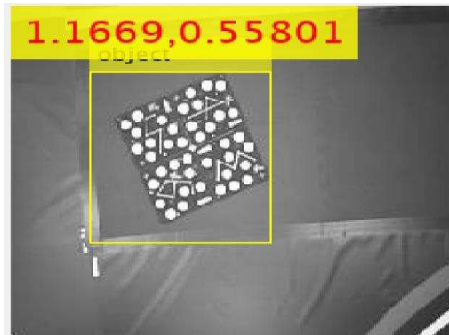


Figure 23: Position Estimation of Detected Object

As can be seen from figure 23 above, the regression model predicts the estimated position of the object after it has been detected and bounding boxes drawn.

2.2.5 Drone Landing

When the detected object coordinates have been received the desired 'x' and 'y' positions from the grid search is overridden and the object coordinates become the desired 'x' and 'y' positions of the QDrone. When the current position of the QDrone is equal to the desired position, the land signal is triggered. The screenshot below shows the design of the landing system:

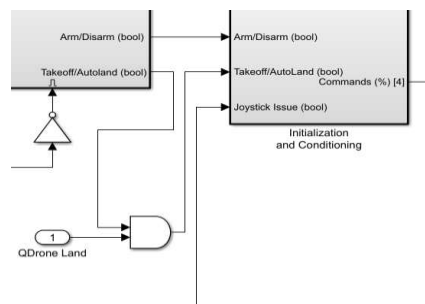


Figure 24: QDrone Landing System

As can be seen from 24 above, when the land signal goes to '0' the AND gate outputs '0' which triggers QDrone landing.

2.3 Implementation

The goal of the project was achieved as the QDrone can search an area iteratively, detect an object and land on it. The screenshots below show a timelapse for the QDrone searching detection and landing:



Figure 25: System Implementation

As can be seen from figure 25 above, the QDrone searched the area until it detected the object and landed on it. In order to implement the system, the proper integration of each block had to be carefully designed. The figure below system implementation design:

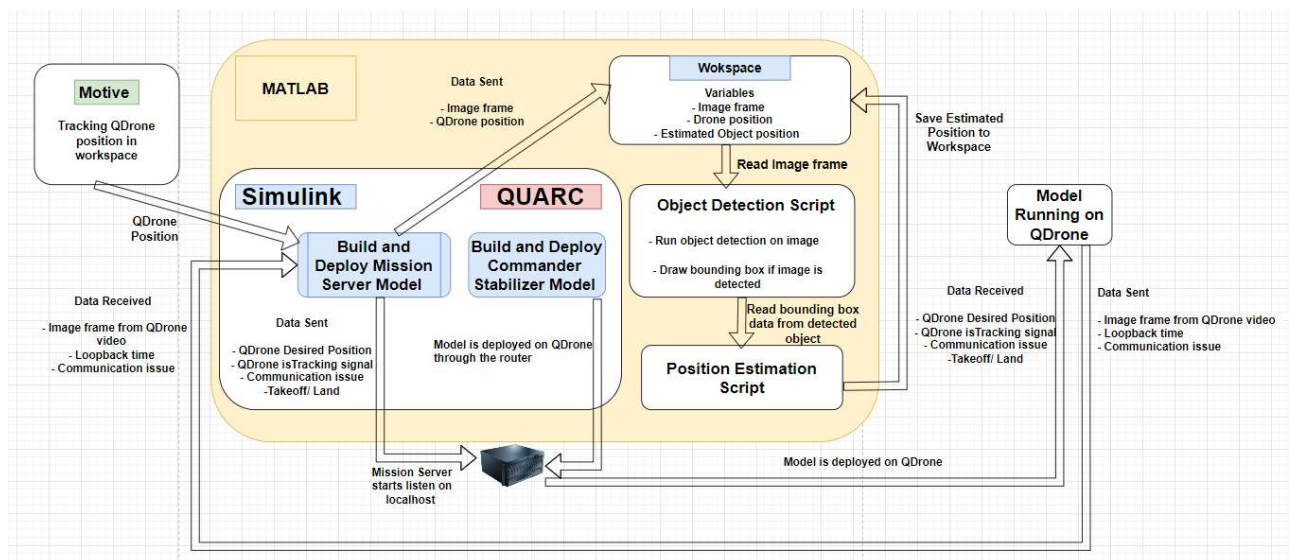


Figure 26: System Implementation Architecture

As can be seen from figure 26 above the system was loosely coupled and well integrated. The mission server model which is deployed on the host machine fetched the position of the QDrone from the Motive software through its API. It then starts to listen on its port in the router for connection requests. When the Command stabilizer is built and deployed, it connects the URI of the host machine and starts receiving data. The Mission Server sends instruction to the command stabilizer to start flying the QDrone. During the Grid Search, live video is also streamed frame-by-frame to the Mission Server model which save the

image to the workspace. The Mission Server model also saves the positional information of the QDrone to the workspace. The object detection and position estimation scripts are then started. The object detection script reads the saved image from the workspace and draws bounding boxes if the object is detected in the image. If bounding boxes are drawn, the information is sent to the Position Estimation models which estimate the real-world coordinates of the object and save to the workspace. The Mission server model reads these coordinates and sets it as the desired pose. The Commander Stabilizer navigates the QDrone to this position. When the QDrone current position is equal to the desired position, the Mission Server send the land signal to the Command Stabilizer which lands the QDrone.

The Simulink models were designed to operate at a sample time of 0.001s, the Object Detection and Position Estimation scripts were set to operate at 0.1s sample time. This was done to ensure the system was real-time. However, a bottleneck in this system was that the QDrone camera operates at a maximum sample time of 0.03s. This introduced some data rate mismatch, but it was tackled by using a Rate Transition block. Also, due to memory constraints, the images from the QDrone video feed were resized before being transmitted.

2.4 Testing

Testing was carried out for each system block and on the entire system after integration.

2.4.1 Data Plan

The captured dataset was very crucial to the performance of the system so multiple images were captured at different angles and at different heights. Also, it was important to capture the images for training and testing with the camera of the QDrone as the pixel mapping of a different camera will be different. The dataset used to train the object detection and position estimation models were created in the laboratory.

2.4.2 Validation and Verification

The validation and Verification of each block of the system is described below:

Drone Control and Stabilization: The stability of the QDrone at a specified altitude was verified visually and by saving the positional data of the QDrone over time. This data was then plotted and verified that the QDrone was indeed stable at a constant altitude. The figure below shows this plot:

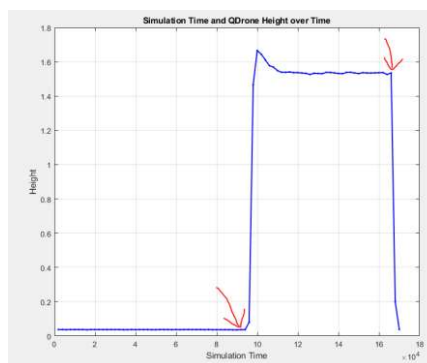


Figure 27: Simulation Time vs QDrone Height

As can be seen from figure 27 above the QDrone was able to maintain a constant altitude as the arrows show when the takeoff and land signals were sent to the QDrone.

Drone Search: The performance of the Grid search algorithm was visually and also plotting the x,y coordinates of the QDrone over time. This plot showed if the entire specified grid area was covered

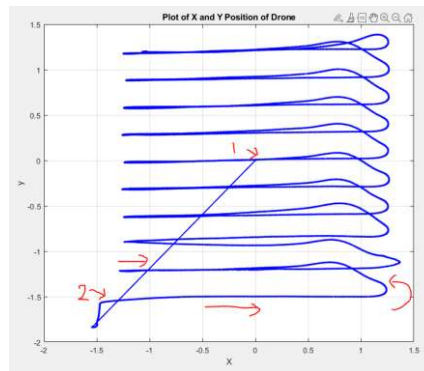


Figure 28: Drone Grid Search

As can be seen from figure 28 above, the QDrone is able to transverse the specified grid area. Arrow 1 indicates the initial position of the QDrone before starting the grid search. The QDrone then navigates to the minimum x and y coordinates where the grid is started.

Object Detection: The object detection model was tested by evaluating the detector with a test set of images. These images consist of images without the object and images with the object at various positions. The figure below shows the result using test images.

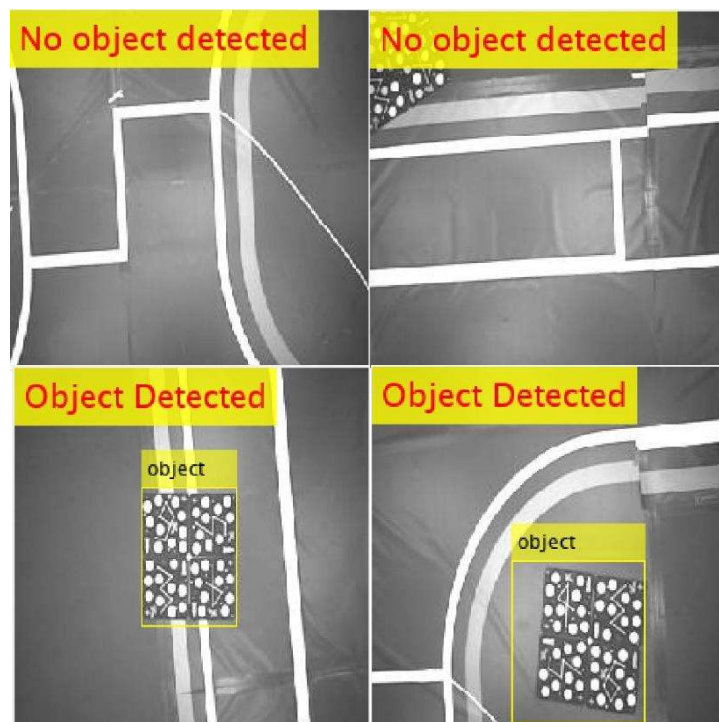


Figure 29: Object Detector Results

As can be seen from figure 29 above, the object detector performed well in detecting the object and drawing bounding boxes on the detected object. The top right image show and an incomplete section of the object in the image. The object detector rightly did not detect it as the entire object needs to be detected. Also, the object detection model was tested by evaluating the Mean Average Precision (MAP) of the detector on a set of validation images. The figure below shows the MAP plot:

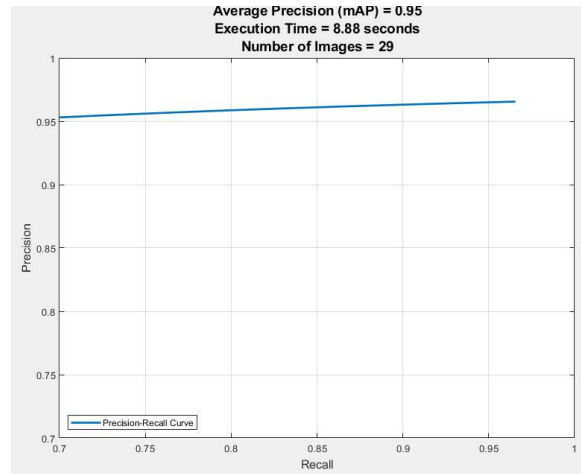


Figure 30: Mean Average Precision Plot

As can be seen from figure 30 above the detector has an average precision of 0.95, this means the detector is accurate. ‘Recall’ which is shown on the x-axis depicts how well the detector detects objects in images while the ‘Precision’ on the y-axis depicts how accurate this detection is.

Position Estimation: The Position Estimation block was tested with images which detected objects and QDrone position. The accuracy of this block was validated by evaluating the performance metric of the regression model.

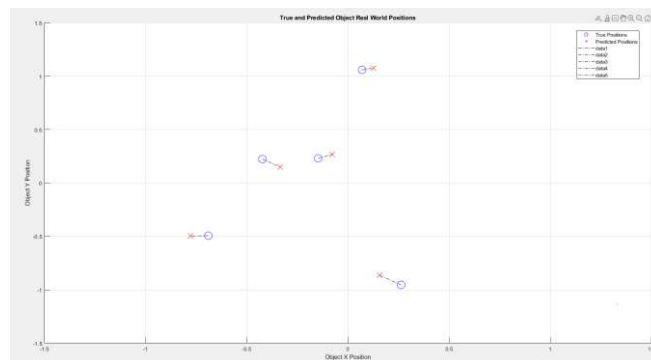


Figure 31: Actual vs Predicted Object Positions

As can be seen from figure 31 above the regression model is able to estimate the real-world coordinates of the object with minimal error. Also, the performance metrics were evaluated for the regression model and fined tuned. The figure below shows the

Linear regression model: $y \sim 1 + x1 + x2 + x3 + x4 + x5 + x6$					Linear regression model: $y \sim 1 + x1 + x2 + x3 + x4 + x5 + x6$				
Estimated Coefficients:					Estimated Coefficients:				
	Estimate	SE	tStat	pValue		Estimate	SE	tStat	pValue
(Intercept)	-1.3793	0.62508	-2.2066	0.034194	(Intercept)	0.86704	0.4557	1.9027	0.065581
x1	1.0282	0.035738	28.77	1.8838e-25	x1	0.053731	0.026054	2.0623	0.046885
x2	0.05427	0.049476	1.0969	0.28039	x2	1.0034	0.036069	27.819	5.6453e-25
x3	-0.015285	0.012043	-1.2692	0.21299	x3	-0.018667	0.0087799	-2.1261	0.040838
x4	0.014898	0.01202	1.2395	0.22366	x4	0.011476	0.0087627	1.3097	0.19909
x5	0.010331	0.014377	0.71855	0.47733	x5	0.018553	0.010481	1.7701	0.085673
x6	-0.0018424	0.014262	-0.12918	0.89798	x6	-0.018984	0.010398	-1.8258	0.076668
Number of observations: 41, Error degrees of freedom: 34 Root Mean Squared Error: 0.168 R-squared: 0.967, Adjusted R-Squared: 0.961 F-statistic vs. constant model: 164, p-value = 1.33e-23					Number of observations: 41, Error degrees of freedom: 34 Root Mean Squared Error: 0.122 R-squared: 0.976, Adjusted R-Squared: 0.971 F-statistic vs. constant model: 226, p-value = 6.72e-26				

Figure 32: Model Coefficients and Performance (X - Left, Y-Right)

As can be seen from figure 32 above, the regression models for predicting the x and y coordinates performed quite well.

3. Overall Results and Analysis

In this section, the project is evaluated based the outcomes, challenges and solutions, possible areas for improvement and potential benefits to career.

3.1 Overall Results

The project was executed successfully as the QDrone was able to autonomously search a specified area, detect an object, estimate its position and land on it. During the project the feasibility and effectiveness of various solution were explored. The images below show the QDrone system in action:

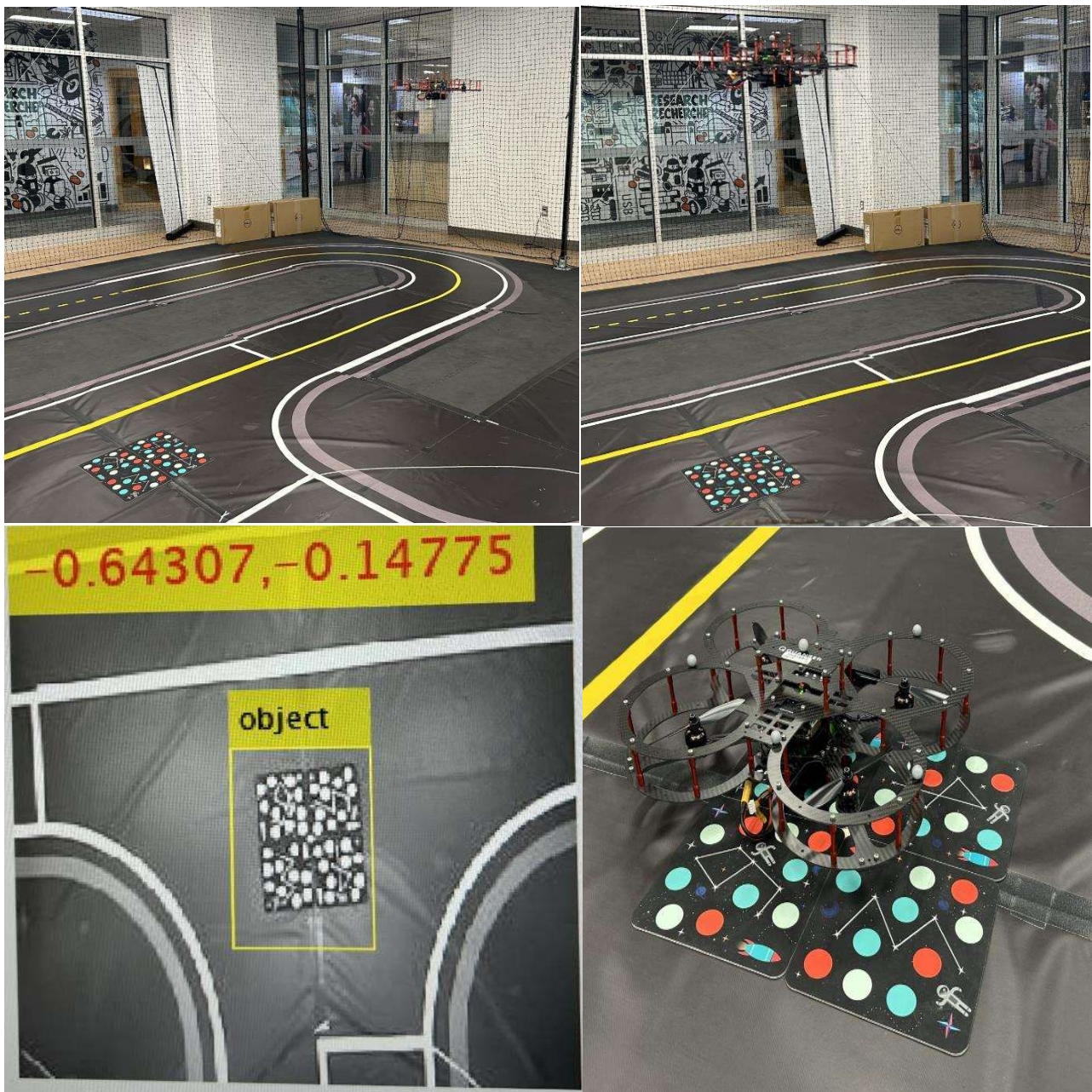


Figure 33: Autonomous Target Detection and Landing

As can be seen from figure 33 above, the system performs well.

3.2 Analysis

Some challenges were encountered during the project and solutions to select an optimal solution. The challenges encountered are listed below:

QDrone Video Transmission: The nominal setting of the QDrone camera was a resolution of 640x480 and a framerate of 30Hz. This presented its unique challenges in the design of the system and several optimizations had to be made.

1. **Image Resize:** The camera resolution of 640x480 was introducing significant latency and memory overhead. The solution implemented was to use an 'Image Resize' Simulink block to reduce the image size before transmission to ensure a balanced trade-off between video quality and transmission.
2. **Send and Receive Buffer:** Also, an appropriate 'Send' and 'Receive' buffer was calculated to ensure images were transmitted seamlessly. The calculation for the 'Send' and 'Receive' buffer size is shown below:

$$\begin{aligned} \text{Send, Receive buffer} &= \text{Image Width} \times \text{Image Height} \times \text{Bits per pixel} \\ \text{Send, Receive buffer} &= 224 \times 224 \times 8 = 401,408 \end{aligned}$$

3. **First In First Out (FIFO):** The 'Send' and 'Receive' FIFO buffer ensures synchronized data flow between the stream server and stream client blocks as size was calculated. The FIFO buffer allows for the collection of data in a queue for transfer. The size of the FIFO should be big enough to allow for the accumulation of new data while the previous data is being sent. The calculation used to calculate the FIFO buffer size is shown below:

$$\begin{aligned} \text{FIFO buffer size} &= \text{Image Width} \times \text{Image Height} \times \text{Queue size} \\ \text{FIFO buffer size} &= 224 \times 224 \times 5 = 250,880 \end{aligned}$$

4. **Data Rate Mismatch:** Furthermore, the Simulink models was designed to operate at a sampling time of 0.001s but the QDrone camera could operate at a maximum sampling time of 0.03. A Rate Transition block which is used to match data transfer between blocks of different sampling time to tackle this issue.

The figure below shows the implemented video transmission system:

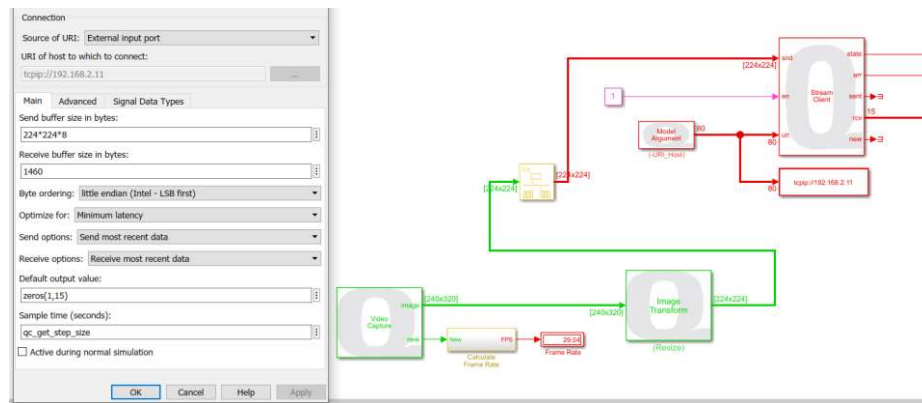


Figure 34: Video Transmission System

3.3 Object Detection

Image classification models were initially used for object detection. These models achieved high accuracies, but they also classified images where the object was partially visible as good images. Also, the image classification models had not system to draw bounding boxes need for the position estimation model. Some time was lost due in the process as all the previous models built using image classification had to be abandoned. The figure below shows the image classification model classifying an image where the object is not fully visible ad ‘good’



Image Classifier False Positive

3.4 Improvement Opportunities

The key areas for improvement in this project are in the AI models and Video transmission. The object detection model’s performance can be improved by using a larger dataset. Approximately 150 images were annotated and used to train the object detector. While this achieved good results, with more training images, the detector can perform better and produce better bounding boxes. Furthermore, for the regression models, the position estimation can be improved by collecting more data to train the regression model. This will improve the precision of the position estimation.

3.5 Learning Outcomes and Career Objectives

As a graduate student of the Applied Artificial Intelligence program, this project has provided deep insights into real-world applications of AI. This will also help in exploring career opportunities

4. Deployment Plan

Since this system was designed with tools that were specially designed for research purposes, the deployment of the project is specific to the Quanser Research Studio environment. In this section, the step to run this project is discussed.

4.1 Requirements

It is assumed that the research environment has already been setup as described in section 2.2.1 and all the tools are present. The files required to run this project are:

1. Mission_Server_DT Simulink Model
2. Command_Stabilizer_DT Simulink Model
3. Object_Position_Estimation Script
4. OptiTrack Calibration Files

4.2 QDrone Power Up and Connectivity

After the QDrone is powered up, open the Netgear Router admin page, and copy the Internet Protocol (IP) address of the QDrone and host machine as indicated in the figure below:

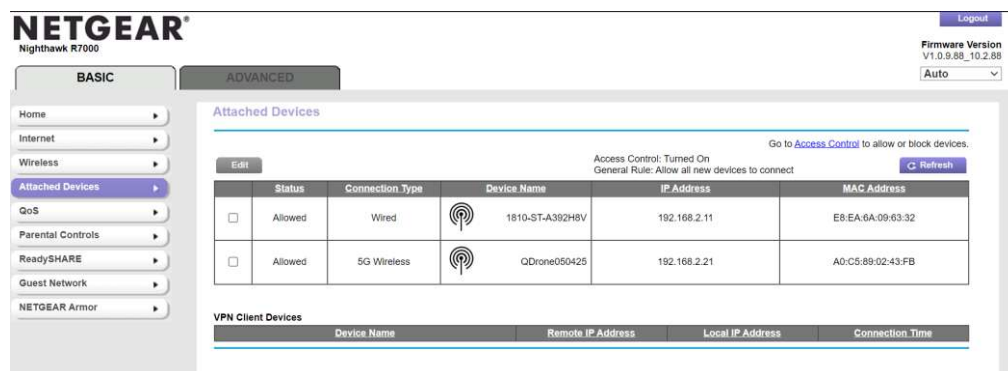


Figure 35: Netgear Router Connected Devices

As can be seen from figure 35, the top IP address corresponds to the IP address of the host machine while the bottom corresponds to the IP address of the QDrone. If after powering the QDrone, the IP address is not visible it indicates there is a connection issue and it is recommended to read the Quanser Research Studio setup manual. Subsequently if the Netgear admin page is not accessible, it means the router is not connected to the network.

4.3 Mission_Server_DT Simulink Model

1. Launch the Mission_Server_DT model in Simulink. Ensure the 'System Target File Description' is set to 'quarc_win64.tlc'. This tell the Simulink that the model will be deployed externally on the host machine.

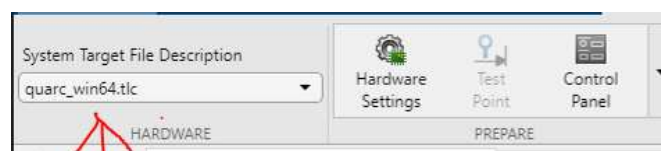


Figure 36: System Target File Description

If “quarc_win64.tlc” is not available, then it means QUARC is not installed or is corrupted. In this case, verify that QUARC is installed and there is an active license.

2. Right-click on the Simulink model and select ‘Model Configuration Properties’. Select ‘Interface’ under ‘Code Generation’ tab. Set the ‘-URI_HOST’ to ‘tcpip://localhost’ as shown in the figure below:

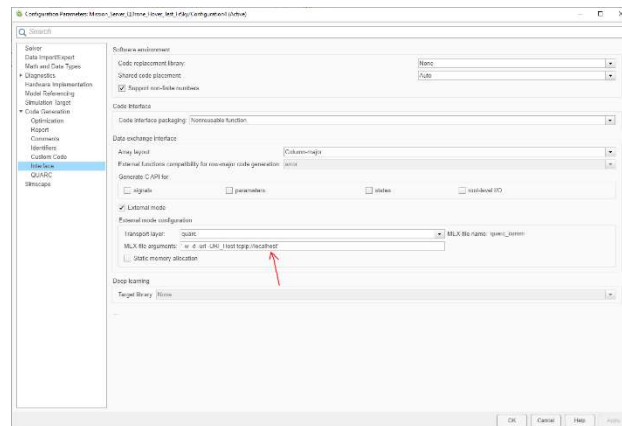


Figure 37: Mex File Arguments

It is important to set the MEX-file as shown in the figure or it will result in errors when the model is run. Click ‘Apply’ then ‘Ok’ to close the window.

3. Ensure the FrSky Controller is powered and connected. This can be verified by confirming the LED of the USB dongle is green. Also ensure the emergency land button of the Controller is in position ‘3’ (Downmost position) and the takeoff/land is in position ‘1’ (Topmost position) as shown in the figure below:



Figure 38: FrSky Controller Switch Positions and USB dongle

4. Double click on the ‘Data Server’ subsystem in the model as show below then double click of the ‘OptiTrack’ block. Upload the OptiTrack calibration files in the window that opens as shown below:

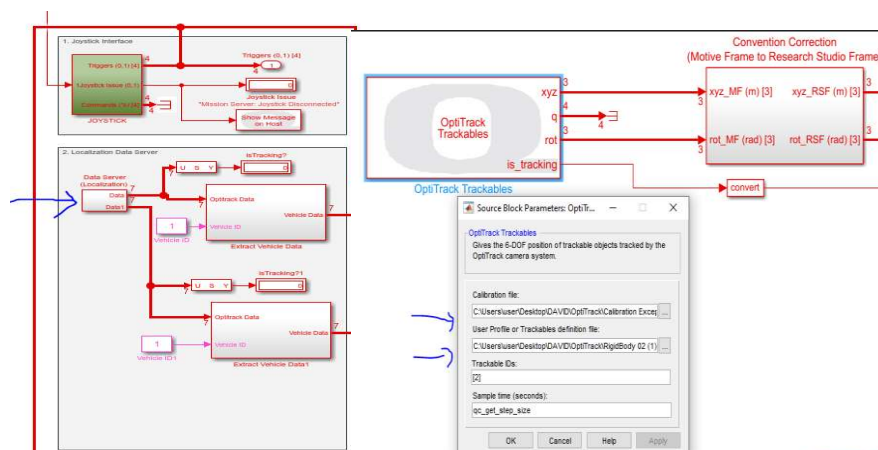


Figure 39: Calibration File Upload

It is important to upload the latest calibration files, or it will result in an error when running the model.

- Click on 'Monitor & Tune' to build and deploy the model. After the build is built and deployed the simulation time at the bottom should start counting as shown below:

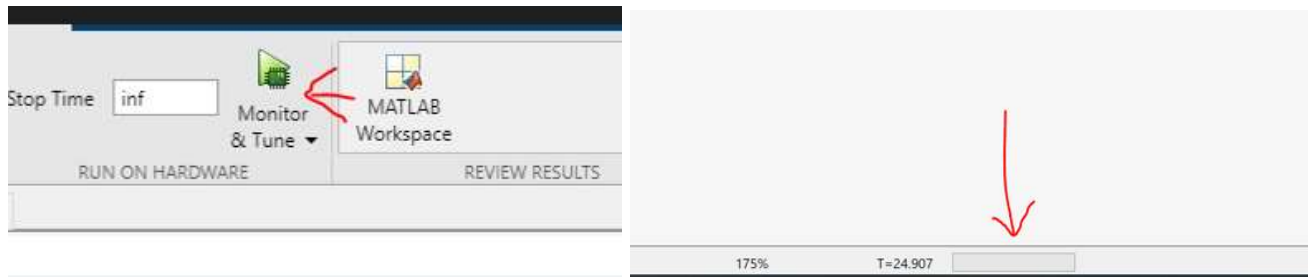


Figure 40: Monitor and Tune

As can be seen the model has been built, deployed, and is running.

- Double click the 'Manual Switch' block to enable the grid search as shown below:

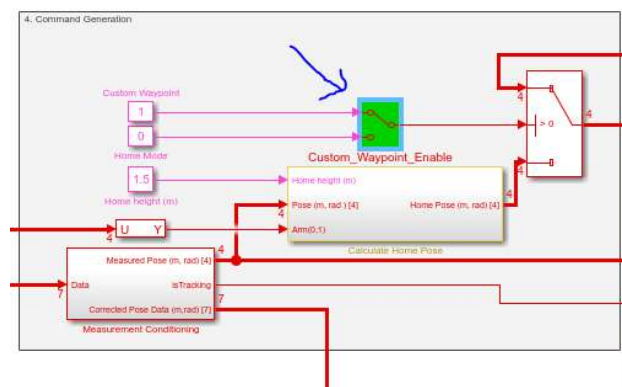


Figure 41: Grid Search Enable

4.4 Command Stabilizer Simulink Model

- Launch the Command_Stabilizer model and verify that the 'System Target File Description' is set to 'quarc_linux_x64.tlc' as shown in the figure below:

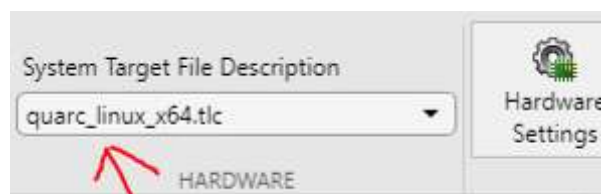


Figure 42: System Target File Description

As can be seen from figure 42 above, the model is set to be deployed on a 64-bit linux system which corresponds to the operating system of the QDrone.

- Right-click on the model and select 'Model Configuration Properties'. Set the '-URI_HOST' of the MEX-File to the IP address of the host and '-uri' to the IP address of the QDrone as shown below:

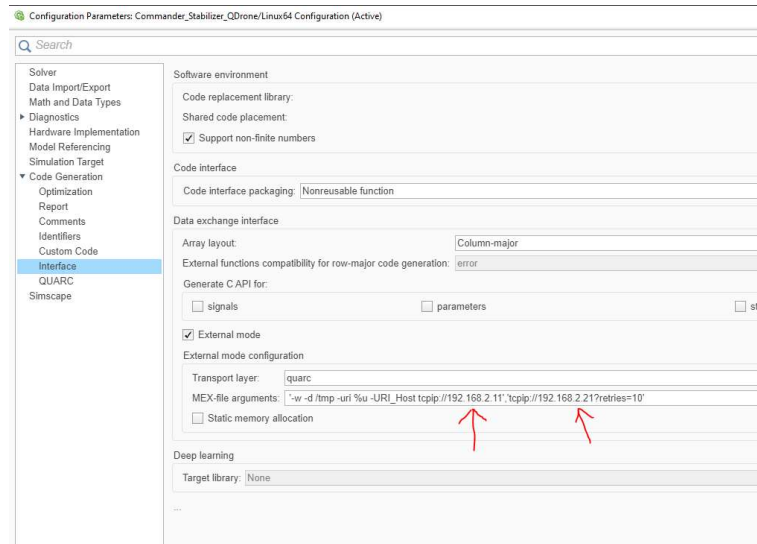


Figure 43: Command_Stabilizer Mex-File Arguments

It is important to set the IP addresses correctly, incorrect IP addresses means the model will not be deployed on the QDrone.

3. Click on 'Monitor&Tune' to build and deploy the model.
4. The control signals of the QDrone will open in a new window. Wait for the 'Initialize' signal to go to '0' before starting the system. Also make sure all signals are 0 as shown in the image below:

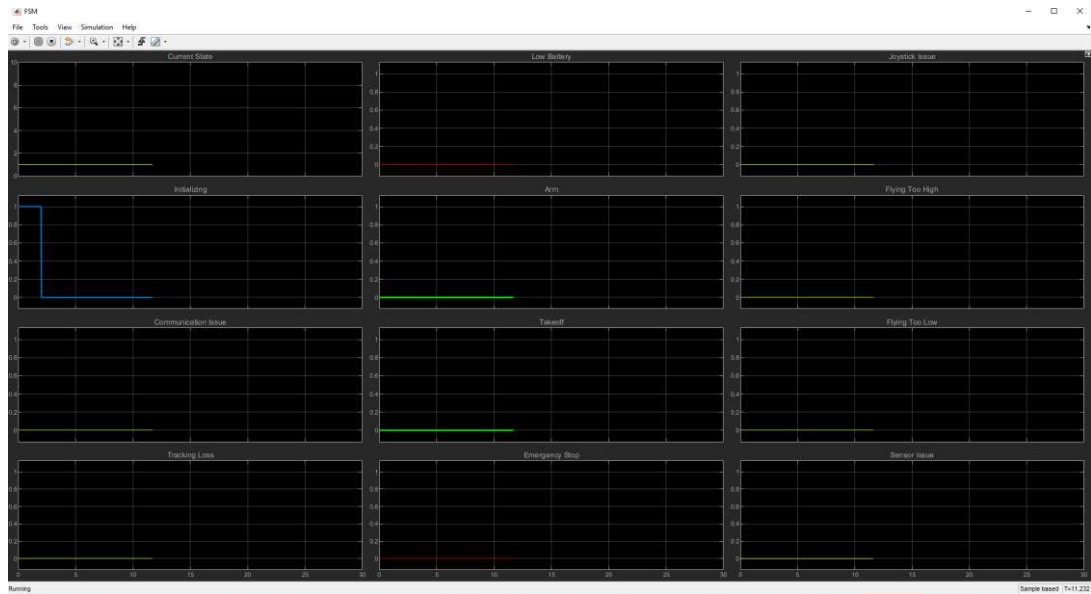


Figure 44: QDrone Signals Monitor

5. Move the FrSky Controller emergency switch to position '1' (Top) to arm the QDrone, the 'Arm' signal should go to '1'. Move the takeoff/land signal to position '2' (Middle) to takeoff the QDrone, subsequently, the takeoff signal should go to '1' as shown below:

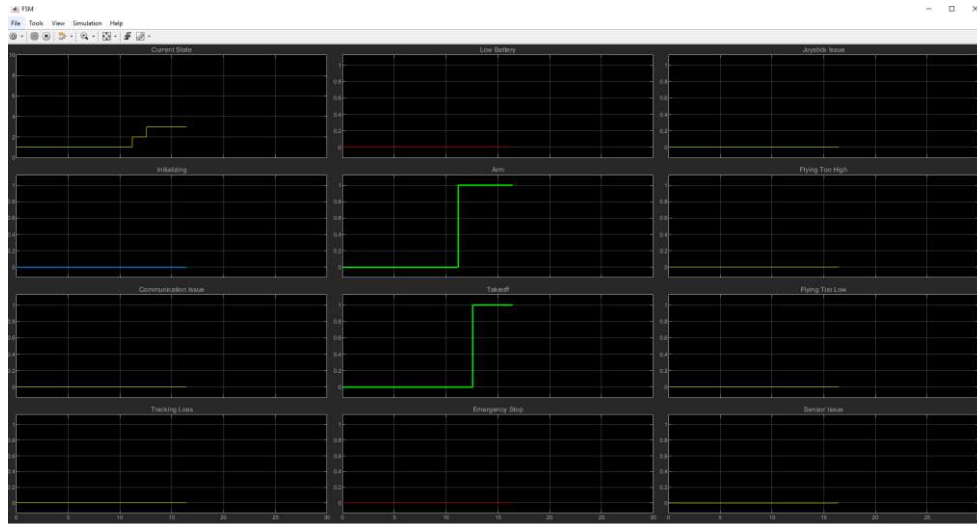


Figure 45: QDrone Arm and Takeoff Signals

6. The QDrone will takeoff and start searching the area.

4.5 Object_Position_Estimation Script

1. Open the object_position_estimation script and click the 'Play' button.
2. This script will start object detection on the images from the QDrone video

5. Conclusion and Future Work

Through this project, it clearly shows that drone can greatly improve SAR operation. This project has a lot of possibility for future improvements. The position estimation was done for stationary object. Future work can include performing position estimation for moving objects. Also, the object detection model can be trained to detect various objects. A suitable candidate for this would be to the MS-coco dataset which has annotation of different objects. An object detection model can train using the MS-Coco dataset as training data.

6. References

- [1] Public Safety Canada. (2022). Quadrennial Search and Rescue Review - Report. 12th August 2022. Retrieved from <https://www.publicsafety.gc.ca/cnt/rsrscs/pblctns/archive-nss-qdrnnl-rvw/index-en.aspx>
- [2] X. Siwen, L. Feng and L. Yanjun, "Pose estimation method for autonomous landing of quadrotor unmanned aerial vehicle," 2022 IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC), Chongqing, China, 2022, pp. 1246-1250, doi: 10.1109/ITOEC53115.2022.9734711.
- [3] X. Siwen, L. Feng and L. Yanjun, "Pose estimation method for autonomous landing of quadrotor unmanned aerial vehicle," 2022 IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC), Chongqing, China, 2022, pp. 1246-1250, doi: 10.1109/ITOEC53115.2022.9734711.
- [4] R. Duan, Y. Guo and P. Lu, "Object Pose Estimation for UAV Navigation Using an End-to-end Lightweight CNN," 2021 China Automation Congress (CAC), Beijing, China, 2021, pp. 2128-2132, doi: 10.1109/CAC53003.2021.9727784.
- [5] R. Duan, Y. Guo and P. Lu, "Object Pose Estimation for UAV Navigation Using an End-to-end Lightweight CNN," 2021 China Automation Congress (CAC), Beijing, China, 2021, pp. 2128-2132, doi: 10.1109/CAC53003.2021.9727784.
- [6] Jason A. Gauthier, "Quadrotor UAV Flight Control with Integrated Mapping and Path Planning Capabilities", Wright State University, Department of Electrical Engineering, 2020. Available Online: https://etd.ohiolink.edu/apexprod/rws_etd/send_file/send?accession=wright1609779989188732&disposition=inline
- [7] Ban Wang, Youmin Zhang, Wei Zhang, "Integrated path planning and trajectory tracking control for quadrotor UAVs with obstacle avoidance in the presence of environmental and systematic uncertainties: Theory and experiment", Aerospace Science and Technology, Volume 120, 2022, 107277, ISSN 1270-9638, <https://doi.org/10.1016/j.ast.2021.107277>.
- [8] Quanser. (2018). QDrone Datasheet. Retrieved July 26, 2023, from <https://www.quanser.com/wp-content/uploads/2018/02/Qdrone-Product-Data-Sheet-v1.2.pdf>
- [9] Quanser. (2018). QDrone Datasheet. Retrieved July 26, 2023, from <https://www.quanser.com/wp-content/uploads/2018/02/Qdrone-Product-Data-Sheet-v1.2.pdf>
- [10] OptiTrack. Rigid Body Tracking. Retrieved July 26, 2023, from <https://docs.optitrack.com/motive/rigid-body-tracking>
- [11] Quanser. (2022). QUARC Datasheet. Retrieved July 26, 2023, from https://download.quanser.com/doc/latest/QUARC_Data_Sheet.pdf
- [12] Quanser. (2022). QUARC Datasheet. Retrieved July 26, 2023, from https://download.quanser.com/doc/latest/QUARC_Data_Sheet.pdf
- [13] MathWorks, "Getting Started with R-CNN, Fast R-CNN, and Faster R-CNN". Retrieved July 27th 2023, from: <https://www.mathworks.com/help/vision/ug/getting-started-with-r-cnn-fast-r-cnn-and-faster-r-cnn.html>
- [14] MathWorks, "Getting Started with R-CNN, Fast R-CNN, and Faster R-CNN". Retrieved July 27th 2023, from: <https://www.mathworks.com/help/vision/ug/getting-started-with-r-cnn-fast-r-cnn-and-faster-r-cnn.html>